

gsDesignNB: Group Sequential Design and Sample Size Re-estimation for Negative Binomial Recurrent Event Trials

Abstract

We present the R package **gsDesignNB** for planning, monitoring, and adapting clinical trials with recurrent event endpoints analyzed using negative binomial models. The package links fixed design sample size calculations, calendar-time group sequential design construction, blinded and unblinded information estimation, sample size re-estimation (SSR), and recurrent-event simulation in a single workflow built on the **gsDesign** package. In the working model, conditional on treatment arm and a subject-level gamma frailty, recurrent events follow a Poisson process with constant within-subject rate, yielding marginal negative binomial counts. For fixed designs, the package extends the methods of Zhu and Lakkis (2014) and Friede and Schmidli (2010) to handle piecewise accrual, exponential dropout, maximum follow-up truncation, and protocol-defined gaps between counted events. Because statistical information for the log rate ratio depends jointly on exposure, event rates, and dispersion, it is not generally proportional to raw event counts. We derive and implement a second-order Taylor correction for Jensen’s inequality bias in the event-gap effective rate; in a 64-scenario simulation sweep, the correction reduces representative approximation error from about 8% to 1% and prevents 1–2.5 percentage points of underpowering when dispersion is moderate-to-high and gaps exceed 20 days. The package also supports calendar-time group sequential monitoring and both blinded and unblinded SSR. In a 13-scenario simulation study, SSR recovers power when nuisance parameters are misspecified, with blinded updates generally requesting larger adapted sample sizes than unblinded updates. The paper emphasizes both the statistical methodology and the software architecture that make these workflows reproducible.

1 Introduction

Recurrent event endpoints are common in clinical trials for conditions such as multiple sclerosis relapses, COPD exacerbations, and heart failure hospitalizations. When every patient has the same underlying event rate, a Poisson model suffices: the variance equals the mean, and information accrues linearly with exposure. Real patient populations, however, exhibit substantial between-patient heterogeneity in event rates. Sicker patients contribute many events while most contribute few, inflating the observed variance well beyond the Poisson mean. Ignoring this overdispersion leads to anticonservative tests and overstated power, understating the sample size needed to detect a clinically meaningful treatment effect.

The negative binomial (NB) model addresses this by adding a single dispersion parameter k that captures the excess variation. Published NB fits from pivotal or large trials confirm that $k > 0$ is the rule rather than the exception: estimates range from roughly 0.8 in asthma (Lipson et al. 2018) to 1.2 in COPD (Wedzicha et al. 2016), 1.5 in heart failure (McMurray et al. 2014), and 3.1 in multiple sclerosis (Hauser et al. 2017). Even moderate dispersion materially reduces the per-patient information — from μ_i under the Poisson to $\mu_i/(1 + k\mu_i)$ under the NB — so that a

design ignoring k can understate the required sample size by 10–30%.

Negative binomial design work also becomes operationally more complicated once a trial moves beyond a fixed final analysis. Information for the log rate ratio depends jointly on exposure, event rates, and dispersion, so raw event counts are only an incomplete proxy for monitoring. Accrual may be piecewise, dropout may truncate follow-up, interim looks may be scheduled on the calendar rather than exactly on information fractions, and some protocols only count events separated by a minimum patient-level gap. In software, these trial features must be represented consistently in planning formulas, recurrent-event simulation, interim data cuts, and alpha-spending calculations.

The **gsDesignNB** package for R provides an integrated toolkit for these tasks:

1. Sample size and power for fixed designs with flexible enrollment, dropout, follow-up truncation, and protocol event gaps.
2. Calendar-time group sequential designs that inherit spending-function infrastructure from the **gsDesign** package (Jennison and Turnbull 1999).
3. Blinded and unblinded information estimation together with blinded and unblinded SSR.
4. Recurrent-event simulation, including seasonal rates, event- and information-driven interim cutting rules, and operating-characteristic summaries.

The contribution of **gsDesignNB** is therefore both scientific and computational. Scientifically, the package implements NB design formulas for settings where information is not proportional to events and introduces a Jensen-correction for event-gap planning. Computationally, it connects planning objects, calendar-time monitoring rules, subject-level simulated trial data, and interim analysis summaries in one workflow. The remainder of the paper first describes that workflow, including the different notions of “gaps” that arise in recurrent-event trials, and then presents the methodological and simulation results that motivated the software design.

2 Package overview and workflow

2.1 Scope and relation to gsDesign

`sample_size_nbinom()` is the fixed-design entry point. It returns a `sample_size_nbinom_result` object containing the planning assumptions, expected exposure, expected events, and variance of the log rate ratio. `gsNBCalendar()` lifts that object into a **gsNB** design that inherits from **gsDesign**, so the NB-specific information calculations live in **gsDesignNB** while spending functions, boundary summaries, and integer rounding remain compatible with the underlying **gsDesign** ecosystem. For simulation, `nb_sim()` and `nb_sim_seasonal()` generate subject-level recurrent-event histories, `cut_data_by_date()` and related helpers convert them to interim analysis data sets, and `sim_gs_nbinom()` or `sim_ssr_nbinom()` produce operating characteristics that can be summarized with `summarize_gs_sim()` and `summarize_ssr_sim()`.

Task	Primary functions	Main outputs
Fixed design planning	<code>sample_size_nbinom()</code> , <code>compute_info_at_time()</code>	<code>sample_size_nbinom_result</code>
Calendar-time GS design	<code>gsNBCalendar()</code> , <code>toInteger()</code> , <code>gsBoundSummary()</code>	gsNB object with information and calendar timing

Task	Primary functions	Main outputs
Recurrent-event data generation	<code>nb_sim()</code> , <code>nb_sim_seasonal()</code>	Subject-level event histories
Interim data construction	<code>cut_data_by_date()</code> , <code>get_cut_date()</code> , <code>get_analysis_date()</code> , <code>cut_completers()</code>	One row per randomized subject with events and exposure
Interim inference and information	<code>mutze_test()</code> , <code>calculate_blinded_info()</code> , <code>estimate_nb_mom()</code>	Wald statistics and blinded/unblinded information estimates
GS operating-characteristic simulation	<code>sim_gs_nbinom()</code> , <code>check_gs_bound()</code> , <code>summarize_gs_sim()</code>	Boundary-crossing summaries
Adaptive simulation	<code>blinded_ssr()</code> , <code>unblinded_ssr()</code> , <code>sim_ssr_nbinom()</code> , <code>summarize_ssr_sim()</code>	Adapted sample sizes and SSR summaries

The package also re-exports the common **gsDesign** spending functions so that users can stay in one namespace when they move from fixed design planning to spending-function specification. Internally, the simulation engines rely on **data.table** for aggregation and on reproducible parallel random number streams for larger Monte Carlo studies.

2.2 Timing, information, and three notions of gaps

The slide deck that motivated this manuscript uses the word “gap” in three different senses, and distinguishing them is important for both the methods and the software.

Concept	Scale	Role in the package
Analysis gap Δ_i	Trial calendar	Minimum time between database locks or interim looks
Timing heterogeneity	Accrual, dropout, and follow-up	Determines how exposure and information accumulate over calendar time
Event gap g	Patient time	Defines which recurrent events count and how much post-event time is removed from risk

The first two affect *when* an analysis can occur. The third affects *which* events contribute to the endpoint and how much at-risk exposure remains after each counted event. In **gsDesignNB**, the same `event_gap` argument is carried through planning, simulation, and interim data cutting, so the effective event rate, counted events, and at-risk exposure stay aligned. By contrast, calendar-time analysis constraints are handled through `analysis_times` in `gsNBCalendar()` or through the search criteria in `get_cut_date()`.

2.3 Primary and secondary workflows

For most users, the package workflow is:

1. Use `sample_size_nbinom()` to build a fixed design under the planned rates, dispersion, accrual, dropout, and follow-up assumptions.

2. Convert that result to a group sequential design with `gsNBCalendar()`, optionally rounding with `toInteger()`.
3. Choose the appropriate simulator: `sim_gs_nbinom()` for calendar-time GS verification or `sim_ssr_nbinom()` for adaptive simulations with blinded or unblinded SSR.
4. Summarize operating characteristics with `check_gs_bound()`, `summarize_gs_sim()`, and `summarize_ssr_sim()`.

The package also includes several secondary workflows that are important in practice but do not change the core architecture.

Additional capability	Main interface	Purpose
Seasonal recurrent-event simulation	<code>nb_sim_seasonal()</code>	Time-varying event rates with season-specific intervals
Completer-driven interims	<code>cut_date_for_completers()</code> , <code>cut_completers()</code>	Interim looks keyed to completed follow-up rather than only calendar time or events
Non-inferiority / super-superiority	<code>sample_size_nbinom(rr0 = ...)</code>	Alternative null hypotheses in the same design framework
Blinded-information diagnostics	<code>calculate_blinded_info()</code> , <code>estimate_nb_mom()</code>	Stability checks for sparse or unstable interims
Interactive review	<code>run_ssr_shiny()</code>	Shiny wrapper around the SSR engine for exploratory use

3 Sample size methodology

3.1 Negative binomial model

For subject i in treatment group g with exposure time t_i , the event count Y_i follows a negative binomial distribution with mean $\mu_i = \lambda_g t_i$ and variance $\text{Var}(Y_i) = \mu_i + k\mu_i^2$, where λ_g is the group-specific event rate and k is the dispersion parameter. This arises from a Gamma–Poisson mixture: each subject has a latent rate $\Lambda_i \sim \text{Gamma}(1/k, k\lambda_g)$ and generates events from $\text{Poisson}(\Lambda_i t_i)$.

3.2 Wald test and sample size formula

The treatment effect is the log rate ratio $\theta = \log(\lambda_2/\lambda_1)$, estimated via maximum likelihood under a negative binomial GLM with log link and exposure offset. The design target is statistical

information, $\mathcal{J} = 1/\text{Var}(\hat{\theta})$, not the raw event count alone. The Fisher information per subject is $\mathcal{J}_i = \mu_i/(1 + k\mu_i)$. If $W_g = \sum_{i \in g} \mu_i/(1 + k_g\mu_i)$, then

$$\mathcal{J} = \frac{1}{1/W_1 + 1/W_2}.$$

When exposures are summarized by arm-specific averages and the randomization ratio is $r = n_2/n_1$, this leads to the fixed-design sample size formula

$$n_1 = \frac{(z_{\alpha/s} + z_{\beta})^2}{(\theta - \theta_0)^2} \left[\left(\frac{1}{\mu_1} + k_1 \right) + \frac{1}{r} \left(\frac{1}{\mu_2} + k_2 \right) \right]$$

where $\mu_g = \lambda_g \bar{t}_g$, $\bar{t}_g$ is the average exposure, and k_g may be inflated by a variance correction factor $Q_g = E[t_g^2]/(E[t_g])^2$ to account for variable follow-up (Zhu and Lakkis 2014). Accordingly, two analyses with similar event totals can carry different information if follow-up or dispersion differs.

This formula is equivalent across the three primary references: Method 3 of Zhu and Lakkis (2014) (fixed sample size), Friede and Schmidli (2010) (blinded SSR), and Mütze et al. (2019) (group sequential designs). The practical difference from more familiar first-event formulas is that when the protocol imposes an event gap, the expected count μ_g must reflect the gap-corrected effective rate rather than a naive event-count proxy.

In `sample_size_nbinom()`, the same information formula is used for superiority, non-inferiority, and super-superiority designs via the `rr0` argument, and the nuisance parameters may be specified either as common scalars or as arm-specific vectors.

3.3 Jensen’s inequality correction for event gaps

In trials with a mandatory gap of duration γ after each event (e.g., a recovery period), the effective rate is commonly approximated as $\lambda_{\text{eff}} = \lambda/(1 + \lambda\gamma)$. However, this is a population-level average that ignores subject-level heterogeneity. Since $f(x) = x/(1 + x\gamma)$ is concave and subject rates Λ_i are random (Gamma-distributed), Jensen’s inequality gives $E[f(\Lambda)] < f(E[\Lambda])$.

We derive a second-order Taylor correction:

$$\lambda_{\text{eff}} \approx \frac{\lambda}{1 + \lambda\gamma} \left(1 - \frac{k\lambda\gamma}{(1 + \lambda\gamma)^2} \right)$$

using $f''(\lambda) = -2\gamma/(1 + \lambda\gamma)^3$ and $\text{Var}(\Lambda) = k\lambda^2$. Table 4 shows the correction magnitude.

Table 4: Jensen correction magnitude for selected parameter combinations.

λ	k	γ	Correction (%)
1.0	1.0	1.0	25.0
1.0	1.0	0.5	22.2
0.5	1.0	1.0	22.2
0.3	1.0	1.0	17.8
0.5	1.0	0.5	16.0

λ	k	γ	Correction (%)
1.0	0.5	1.0	12.5
0.3	1.0	0.5	11.3
1.0	0.5	0.5	11.1

A simulation study with 10,000 replicates per scenario confirmed that the corrected formula reduces the effective rate estimation error from approximately 8% to 1% and produces slightly conservative power (Table 5). During package development, this correction was needed for simulation-based power to agree with the analytical planning formula when event gaps were non-negligible.

In software terms, the same gap rule is used by `nb_sim()` and `cut_data_by_date()`: events that fall inside the post-event gap are not counted, and the corresponding gap time is removed from the at-risk exposure used for information calculations.

Table 5: Power comparison: Jensen-corrected vs naive formula (10,000 replicates each).

k	Gap (days)	n (corrected)	n (naive)	Power (corr.)	Power (naive)
0.5	20	436	422	0.9061	0.8992
1.0	20	712	684	0.9170	0.9033
0.5	30	454	436	0.9042	0.8934
1.0	30	740	700	0.9105	0.8988

3.3.1 Broader parameter sweep

The four scenarios in Table 5 fix $\lambda_1 = 0.4$ and vary only k and the gap duration. To characterize *when* the correction matters across a wider range of clinically relevant settings, we conducted a broader simulation sweep over three axes: the control event rate λ_1 , the dispersion k , and the event gap γ .

Parameter grid. The sweep covers $4 \times 4 \times 4 = 64$ scenarios defined by:

Table 6: Broad sweep parameter grid for Jensen correction impact study.

Parameter	Values	Clinical rationale
λ_1 (events/month)	0.3, 0.5, 1.0, 2.0	Heart failure hospitalizations (\$ 0.3) <i>through</i> COPD exacerbations (0.5) <i>to</i> MS relapses (\$1–2)
k (dispersion)	0.1, 0.3, 0.5, 1.0	Near-Poisson (0.1) to substantial patient heterogeneity (1.0); published NB trials typically report $k \in [0.3, 1.0]$
Gap (days)	10, 20, 30, 45	Short recovery (migraine) to extended gap (steroid taper in COPD exacerbations)

The remaining design parameters are held constant across all scenarios: $\lambda_2/\lambda_1 = 0.7$ (30% rate reduction), power = 90%, one-sided $\alpha = 0.025$, dropout rate = 0.1/12/month, maximum follow-up = 12 months, trial duration = 24 months, and piecewise accrual (rate R for 0–6 months, $2R$ for

6–12 months). These are fixed because they affect power symmetrically for both the corrected and naive designs, so they shift both power estimates together without meaningfully changing the *gap* between them. The three chosen axes (λ_1 , k , γ) correspond directly to the three terms in the correction factor:

$$1 - \frac{k\lambda_1\gamma}{(1 + \lambda_1\gamma)^2}$$

Rationale for disease-area calibration. The parameter ranges are informed by published recurrent event trials. Heart failure hospitalization trials such as PARADIGM-HF report control rates of approximately 0.3 events/month with dispersion near 0.5 (McMurray et al. 2014). COPD exacerbation trials (e.g., FLAME, IMPACT) report rates around 0.5–0.8/month with $k \in [0.5, 1.0]$ and commonly define a new exacerbation as requiring \$28 *symptom-free* days [Wedzicha 2016, Lipson 2018]. Multiple sclerosis relapse trials observe rates of 1–2/year with moderate to high overdispersion [Hauser 2017, ocrelizumab]. At one extreme ($\lambda_1 = 0.3$, $k = 0.1$, $\text{gap} = 10$ days), the theoretical correction factor is below 0.1% and the naive formula suffices. At the other ($\lambda_1 = 2.0$, $k = 1.0$, $\text{gap} = 45$ days), it exceeds 10%, and the underpowering from the naive formula becomes practically meaningful.

Simulation protocol. For each scenario, we compute both the Jensen-corrected and naive sample sizes, simulate 3,500 trials at the corrected n , and extract results for the first n_{naive} subjects from the same simulated data. This paired design makes the power comparison very precise (SE ≈ 0.5 percentage points at 90% power). The aggregated results — scenario parameters, sample sizes, empirical power, paired power differences, and discordant pair counts — are saved for use in downstream analyses.

Results. Table 7 summarizes selected scenarios from the 64-scenario sweep, ordered by the theoretical correction factor. The corrected design achieves power within simulation error of the 90% target across all 64 scenarios (range: 0.896–0.924), while the naive design falls below 90% in the majority of scenarios where $k \geq 0.5$ and the gap exceeds 20 days.

Table 7: Selected scenarios from the broad sweep (3,500 replicates each). Rows are ordered from largest to smallest correction factor; the last two rows illustrate scenarios where the correction has no effect.

λ_1	k	Gap (days)	n (corr.)	n (naive)	Δn	Power (corr.)	Power (naive)	Δ Power (pp)
1.0	1.0	30	426	406	20	0.911	0.899	1.1
0.5	1.0	45	490	456	34	0.910	0.890	2.1
1.0	1.0	45	446	420	26	0.911	0.896	1.5
2.0	1.0	30	402	388	14	0.911	0.900	1.0
0.5	1.0	30	466	442	24	0.914	0.893	2.1
0.3	1.0	45	542	502	40	0.919	0.903	1.6
2.0	1.0	45	418	402	16	0.911	0.899	1.2
0.3	1.0	30	516	488	28	0.918	0.904	1.4
0.5	0.5	45	300	284	16	0.908	0.891	1.7
0.3	0.5	45	348	332	16	0.911	0.895	1.7
2.0	0.1	45	96	96	0	0.918	0.918	0.0

λ_1	k	Gap (days)	n (corr.)	n (naive)	Δn	Power (corr.)	Power (naive)	Δ Power (pp)
0.3	0.1	10	162	162	0	0.900	0.900	0.0

Three patterns emerge:

1. **The correction is driven by the $k \times \gamma$ interaction.** Dispersion $k \geq 0.5$ combined with gaps ≥ 30 days consistently produces 1–2 percentage points of underpowering from the naive formula, corresponding to 14–40 additional subjects (3–8% of total n).
2. **High event rates do not amplify the power gap as much as high dispersion.** At $k = 0.3$, even $\lambda_1 = 2.0$ with a 45-day gap shows only \$0.6 pp of underpowering. Conversely, at $k = 1.0$, $\lambda_1 = 0.5$ with a 30-day gap shows 2.1 pp — the largest gap in the sweep. This is because the correction factor $k\lambda\gamma/(1 + \lambda\gamma)^2$ saturates in λ but grows linearly in k .
3. **The correction is negligible when either k or γ is small.** All scenarios with $k = 0.1$ show $\Delta n \leq 4$ and $\Delta\text{Power} < 0.8$ pp. Similarly, all 10-day gap scenarios show $\Delta n \leq 10$ regardless of k .

These findings provide practical guidance: the Jensen correction is most valuable for trials in therapeutic areas with substantial patient heterogeneity ($k \geq 0.5$) and protocol-defined event gaps exceeding 20 days, such as COPD exacerbation studies with 28-day new-event windows or heart failure trials with extended hospitalization-free windows.

4 Group sequential design, monitoring, and sample size re-estimation

4.1 Calendar-time design construction

The `gsNBCalendar()` function creates a group sequential design from a `sample_size_nbinom_result` by evaluating the planned design at user-specified calendar analysis times. For each look it re-computes expected enrollment, exposure, events, and variance under the planned accrual and follow-up structure, then stores the corresponding statistical information in the `n.I` slot inherited from `gsDesign`. The resulting `gsNB` object therefore behaves like a `gsDesign` object for spending functions and boundary summaries while retaining the NB-specific quantities needed for simulation and interpretation.

This calendar-time emphasis matters because recurrent-event trials are often governed by operational rules rather than by exact information fractions. If C_i denotes a calendar floor for look i , Δ_i a minimum gap after the prior look, and $\mathcal{I}_i$ the target information for that look, the analysis time can be expressed schematically as

$$\tau_i = \max \left\{ C_i, \tau_{i-1} + \Delta_i, \inf \left\{ t : \hat{\mathcal{I}}(t) \geq \mathcal{I}_i \right\} \right\}.$$

`gsNBCalendar()` addresses the planning side of this problem through `analysis_times`, while `get_cut_date()` applies the same logic to simulated or observed trial data by combining calendar, event-count, completer, and information targets. This is the package’s bridge between textbook information-time group sequential design and the calendar rules encountered in real monitoring charters.

4.2 Interim data cuts and information estimates

The function `cut_data_by_date()` converts subject-level recurrent-event histories into an interim analysis data set with one row per randomized subject, containing counted events, time at risk, and total follow-up. The same interface is implemented for both `nb_sim()` and `nb_sim_seasonal()`. `get_analysis_date()` solves the special case of event-driven looks, whereas `cut_date_for_completers()` and `cut_completers()` support analyses defined by the number of subjects who have completed their planned follow-up. More generally, `get_cut_date()` takes the maximum of whichever criteria are supplied, allowing hybrid rules such as “no earlier than month 8 and only after 60 blinded information units.”

Because information depends on exposure and dispersion, the package distinguishes raw event totals from the quantities actually used for monitoring. `calculate_blinded_info()` implements the pooled negative binomial fit of Friede and Schmidli (2010), then splits the pooled rate according to the planned rate ratio. `estimate_nb_mom()` supplies method-of-moments estimates that are useful when maximum likelihood fits become unstable in sparse or highly overdispersed interim data.

4.3 Wald inference, Poisson fallback, and diagnostics

For unblinded interim analyses, `mutze_test()` fits the negative binomial log-linear model with an offset for exposure and returns the Wald statistic for the treatment effect. This is the basic analysis model used throughout the package’s GS simulations. When the fitted NB shape parameter θ is either extremely large (near-Poisson data) or extremely small (numerically unstable overdispersion), `mutze_test()` falls back to a Poisson Wald analysis using the user-set `poisson_threshold`. This fallback is not meant as a theoretical replacement for the NB model; it is a defensive device that keeps simulations and diagnostics well behaved when the ML NB fit is not reliable.

The simulation engines expose several information paths so that users can stress-test an interim monitoring strategy before locking it into a design. In particular, `sim_gs_nbinom()` records unblinded and blinded information under both maximum likelihood and method-of-moments estimates. These alternative columns are diagnostics rather than different estimands: they help show whether a design’s monitoring behavior is robust to the nuisance-parameter estimation method used at the interim.

4.4 Blinded and unblinded SSR

At interim analyses, nuisance parameters (k and the underlying event rates) can be re-estimated to update the required sample size. `blinded_ssr()` uses the pooled model of Friede and Schmidli (2010), preserving masking but typically producing a more conservative adapted sample size. `unblinded_ssr()` instead fits treatment-specific rates and a common dispersion to the interim data, while holding the target treatment effect at its planned value for the final sample size calculation.

`sim_ssr_nbinom()` wraps these choices into a full adaptive simulation engine. It supports “No adaptation”, “Blinded SSR”, and “Unblinded SSR” strategies; searches for interim looks using information-based timing; and allows efficacy and futility bounds to be driven by alternative information measures through `bound_info = "unblinded_ml", "blinded_ml", "unblinded_mom",` or `"blinded_mom"`. In the simulation study below, bounds are computed with spending functions using `usTime = lsTime = min(planned IF, actual IF)` at the interim so that spending is not accelerated when more information than expected is observed. Futility assessment is deferred until

at least 30% of the planned information is available.

From a computing perspective, both `sim_gs_nbinom()` and `sim_ssr_nbinom()` are designed for large Monte Carlo studies: they use `data.table` for repeated aggregation, support reproducible parallel execution, and return both trial-level and analysis-level summaries so that design calibration can be checked after the fact.

5 Reproducible workflows

5.1 Example 1: fixed design to calendar-time GS simulation

The first workflow shows the main path from fixed-design planning to a calendar-time GS simulation. The full executable version appears in the group-sequential simulation vignette; the code here is a minimal example.

```
library(gsDesignNB)

enroll_rate <- data.frame(rate = 12, duration = 6)
fail_rate <- data.frame(
  treatment = c("Control", "Experimental"),
  rate = c(0.6, 0.42),
  dispersion = c(0.4, 0.4)
)
dropout_rate <- data.frame(
  treatment = c("Control", "Experimental"),
  rate = c(0.05, 0.05),
  duration = c(12, 12)
)

fixed_design <- sample_size_nbinom(
  lambda1 = 0.6,
  lambda2 = 0.42,
  dispersion = 0.4,
  power = 0.8,
  alpha = 0.025,
  accrual_rate = enroll_rate$rate,
  accrual_duration = enroll_rate$duration,
  trial_duration = 12,
  max_followup = 6
)

gs_design <- gsNBCalendar(
  fixed_design,
  k = 3,
  test.type = 4,
  analysis_times = c(4, 8, 12)
)

sim_gs <- sim_gs_nbinom(
```

```

n_sims = 50,
enroll_rate = enroll_rate,
fail_rate = fail_rate,
dropout_rate = dropout_rate,
max_followup = 6,
n_target = ceiling(utils::tail(gs_design$n_total, 1)),
design = gs_design,
cuts = lapply(gs_design$T, function(t) list(planned_calendar = t)),
seed = 20260413
)

oc <- summarize_gs_sim(
  check_gs_bound(sim_gs, gs_design, info_col = "info_unblinded_ml")
)

oc$analysis_summary

```

This example illustrates the main design principle of the package: the fixed design determines the target information, `gsNBCalendar()` maps that information to calendar looks, `sim_gs_nbinom()` generates trial histories consistent with the planning assumptions, and `check_gs_bound()` applies the group sequential boundaries on the selected information scale.

5.2 Example 2: information-based SSR

The second workflow shows how the same design object can be used in an adaptive simulation with blinded and unblinded sample size re-estimation.

```

library(gsDesignNB)

ssr_res <- sim_ssr_nbinom(
  n_sims = 50,
  enroll_rate = enroll_rate,
  fail_rate = fail_rate,
  dropout_rate = dropout_rate,
  max_followup = 6,
  design = gs_design,
  strategies = c("No adaptation", "Blinded SSR", "Unblinded SSR"),
  bound_info = "unblinded_ml",
  seed = 20260413
)

summarize_ssr_sim(ssr_res)$trial_summary[
  , c("strategy", "rejection_rate", "mean_n", "pct_adapted")
]

```

The paper's 13-scenario SSR study uses this same engine with a much larger scenario grid, more replicates, and additional design constraints on when adaptation is allowed. The essential software idea, however, is already visible in the compact example: one `gsNB` planning object can drive both non-adaptive and adaptive operating-characteristic evaluations.

5.3 Additional workflows

The package includes further worked examples that round out the software contribution.

Workflow	Main functions	What it adds
Seasonal recurrent-event simulation	<code>nb_sim_seasonal()</code> , <code>cut_data_by_date()</code>	Calendar-time variation in failure rates
Completer-based monitoring	<code>cut_date_for_completers()</code> , <code>cut_completers()</code>	Interim looks keyed to completed follow-up
Non-inferiority and super-superiority	<code>sample_size_nbinom(rr0 = ...)</code>	Alternative null hypotheses in the same design framework
Blinded-information diagnostics	<code>calculate_blinded_info()</code> , <code>estimate_nb_mom()</code>	Stability checks for sparse or unstable interims
Verification studies	<code>nb_sim()</code> , <code>mutze_test()</code> , <code>sim_gs_nbinom()</code>	Simulation-based agreement checks for design formulas

These workflows are documented in the package vignettes so that the manuscript examples can remain brief while the installed package still serves as the full replication resource.

6 Simulation study

6.1 Design

We evaluated `sim_ssr_nbinom()` across 13 scenarios varying the control rate (λ_1), dispersion (k), accrual rate, and rate ratio ($RR = 0.7, 0.8, 0.9, 1.0$). The planned design was built with `sample_size_nbinom()` and `gsNBCalendar()`, targeting 90% power for $RR = 0.7$ with $k = 0.3$, $\lambda_1 = 0.5/\text{month}$, and accrual of 30/month over 12 months. Each non-null scenario used 3,600 simulations; the null scenario used 20,000.

Three strategies were compared: (1) no adaptation, (2) blinded SSR, and (3) unblinded SSR, all using a group sequential design with Hwang–Shih–DeCani spending functions.

6.2 Results

The key findings from the simulation study are:

- **Type I calibration:** Under the null ($RR = 1.0$, misspecified nuisance parameters), the main vignette configuration showed slight inflation at nominal one-sided 2.5%; a sensitivity rerun at one-sided 2.0% improved calibration.
- **Power recovery at $RR = 0.7$:** When the planned rate ratio was correct, both SSR methods recovered power from as low as 60% (no adaptation, worst case) to approximately 85–90%, by increasing the sample size up to $2\times$ planned.
- **Diminishing returns at smaller effects:** For $RR = 0.8$ and $RR = 0.9$, the planned sample size (designed for $RR = 0.7$) was inherently insufficient. SSR compensated for nuisance parameter misspecification but could not overcome the smaller treatment effect.
- **Blinded vs unblinded:** Both methods recovered power, but blinded SSR generally requested larger adapted sample sizes than unblinded SSR because pooled nuisance updates were more

conservative. When operationally acceptable, unblinded SSR may therefore be preferable for efficiency.

7 Discussion

The **gsDesignNB** package fills a gap in the R ecosystem for planning and adapting recurrent-event trials with NB outcomes. Its statistical contributions are the alignment of fixed-design planning, calendar-time monitoring, and sample size re-estimation around Fisher information for the log rate ratio, together with the Jensen correction for event gaps and the accompanying simulation evidence. Its computing contributions are the object flow from `sample_size_nbinom_result` to **gsNB** to simulation outputs; subject-level recurrent-event simulation with event-gap-aware interim cuts; diagnostic information calculations under ML and method-of-moments nuisance estimation; and reproducible Monte Carlo engines for both fixed and adaptive monitoring strategies.

The Jensen correction is important precisely because the software is designed to keep planning and simulation on the same scale. When event gaps are non-negligible, the naive renewal approximation overstates the effective rate and therefore overstates information. The 64-scenario sweep confirms that the correction is most valuable when dispersion and event-gap duration are both substantial: at $k \geq 0.5$ with gaps ≥ 30 days, the naive formula underpowers by about 1–2 percentage points, whereas at $k = 0.1$ or gaps ≤ 10 days the correction adds little to the required sample size. Because the same gap rule is carried into simulation and interim cutting, users can see directly when the correction matters for operating characteristics rather than only for algebra.

The SSR study likewise shows how the package’s computational layer supports statistical conclusions. When the treatment effect is as planned, both blinded and unblinded SSR recover power lost to nuisance-parameter misspecification. Blinded updates preserve masking but tend to request larger adapted sample sizes; unblinded updates are more sample-efficient when operationally acceptable. The ability to compare `bound_info` choices, ML versus method-of-moments information paths, and Poisson fallback behavior inside the same simulation engine is a practical advantage of the package design.

From a software perspective, **gsDesignNB** is complementary to both **gsDesign** and **gscounts**. The general-purpose **gsDesign** package provides mature spending-function and boundary calculations but does not itself supply NB recurrent-event planning formulas, calendar-time translation of those formulas, or recurrent-event simulation with blinded information and SSR. The **gscounts** implementation motivated by Mütze et al. (2019) addresses group sequential NB testing, whereas **gsDesignNB** emphasizes the broader workflow needed in practice: piecewise accrual and dropout, maximum follow-up, protocol event gaps with Jensen correction, seasonal simulation, completer- or information-driven interim timing, and blinded as well as unblinded SSR simulation.

The package’s secondary workflows matter for a software paper even when they are not the main empirical case study. Seasonal simulation extends the constant-rate gamma-frailty model to a common time-varying setting; completer-based cuts capture another operational monitoring rule; and the non-inferiority, blinded-diagnostics, and verification vignettes show that the same core objects support multiple design questions. In that sense, the package is not only a collection of isolated functions but a coherent interface for recurrent-event trial planning and evaluation.

Although the development of the package used modern interactive computing tools, the contribution of interest for a statistical software journal is the reproducible software artifact itself: the open

source implementation, documented API, worked vignettes, and simulation studies that demonstrate how the methods behave under realistic trial scenarios.

8 Reproducibility and software availability

The **gsDesignNB** package is available on GitHub at <https://github.com/keaven/gsDesignNB> with documentation at <https://keaven.github.io/gsDesignNB/>. The package requires $R \geq 4.1.0$ and depends primarily on **gsDesign**, **data.table**, **simtrial**, and **MASS**.

The source distribution includes vignettes for fixed-design NB sample size calculation, calendar-time GS simulation, end-to-end SSR, the larger SSR simulation study, blinded-information diagnostics, seasonal simulation, completer-based interims, non-inferiority design, and verification of the planning formulas against simulation. The two code examples in this paper are deliberately small extracts of those vignettes so that the manuscript highlights the core software interface while the installed package retains more extensive executable workflows. For local exploratory use, the package also provides `run_ssr_shiny()` as an interactive front end to the SSR engine and `preview_pkgdown_site()` for documentation preview.

References

- Friede, T., and Schmidli, H. (2010), “Blinded sample size reestimation with negative binomial counts in superiority and non-inferiority trials,” *Methods of Information in Medicine*, 49, 618–624. <https://doi.org/10.3414/ME09-02-0060>.
- Hauser, S. L., Bar-Or, A., Comi, G., Giovannoni, G., Hartung, H.-P., Hemmer, B., Lublin, F., Montalban, X., Rammohan, K. W., Selmaj, K., and others (2017), “Ocrelizumab versus interferon beta-1a in relapsing multiple sclerosis,” *New England Journal of Medicine*, 376, 221–234. <https://doi.org/10.1056/NEJMoa1601277>.
- Jennison, C., and Turnbull, B. W. (1999), *Group sequential methods with applications to clinical trials*, CRC Press.
- Lipson, D. A., Barnhart, F., Brealey, N., Brooks, J., Criner, G. J., Day, N. C., Dransfield, M. T., Halpin, D. M. G., Han, M. K., Jones, C. E., and others (2018), “Once-daily single-inhaler triple versus dual therapy in patients with COPD,” *New England Journal of Medicine*, 378, 1671–1680. <https://doi.org/10.1056/NEJMoa1713901>.
- McMurray, J. J. V., Packer, M., Desai, A. S., Gong, J., Lefkowitz, M. P., Rizkala, A. R., Rouleau, J. L., Shi, V. C., Solomon, S. D., Swedberg, K., and Zile, M. R. (2014), “Angiotensin–neprilysin inhibition versus enalapril in heart failure,” *New England Journal of Medicine*, 371, 993–1004. <https://doi.org/10.1056/NEJMoa1409077>.
- Mütze, T., Glimm, E., Schmidli, H., and Friede, T. (2019), “Group sequential designs for negative binomial outcomes,” *Statistical Methods in Medical Research*, 28, 2326–2347. <https://doi.org/10.1177/0962280218773115>.
- Wedzicha, J. A., Banerji, D., Chapman, K. R., Vestbo, J., Roche, N., Ayers, R. T., Thach, C., Fogel, R., Patalano, F., and Vogelmeier, C. F. (2016), “Indacaterol–glycopyrronium versus salmeterol–fluticasone for COPD,” *New England Journal of Medicine*, 374, 2222–2234. <https://doi.org/10.1056/NEJMoa1516385>.
- Zhu, H., and Lakkis, H. (2014), “Sample size calculation for comparing two negative binomial rates,” *Statistics in Medicine*, 33, 376–387. <https://doi.org/10.1002/sim.5947>.